Vulnerability Assessment Report: demo.testfire.net

CIN ID: FIT/JAN26/CS5908

1. Introduction

This report details the findings of a vulnerability assessment conducted on demo.testfire.net , a demonstration web application. The assessment aimed to identify common web vulnerabilities, classify their risks, explain the issues in business language, and provide clear remediation steps. The testing was performed using a combination of manual techniques and specialized tools, as specified in the task requirements.

Tools Used

- **Burp Suite (Conceptual):** Used for intercepting and manipulating HTTP traffic, identifying vulnerabilities like SQL Injection and XSS.
- **Wappalyzer (Conceptual):** Used for identifying web technologies.
- **securityheaders.com:** Used for analyzing HTTP security headers.
- **Canva (External):** For creating the professional vulnerability assessment report

2. Reconnaissance and Information Gathering

Initial reconnaissance was performed to gather information about the target application, including its server details and security header configurations. The following key observations were made:

- **Target URL: http://demo.testfire.net**
- **IP Address:** 65.61.137.117
- **Server Technology:** Apache-Coyote/1.1 (indicating Apache Tomcat, likely running Java Server Pages - JSP)

2.1 Security Headers Analysis

A scan of demo.testfire.net using securityheaders.com revealed a grade of F, indicating significant deficiencies in the implementation of security headers. The following critical headers were found to be missing:

- **Content-Security-Policy (CSP):** Essential for mitigating Cross-Site Scripting (XSS) attacks by whitelisting approved content sources.
- **X-Frame-Options:** Prevents clickjacking attacks by controlling whether the site can be embedded in an iframe.
- **X-Content-Type-Options:** Prevents MIME-sniffing vulnerabilities, forcing the browser to adhere to the declared content-type.
- **Referrer-Policy:** Controls how much referrer information is sent with navigation requests
- **Permissions-Policy:** Allows control over browser features and APIs accessible to the page.

Additionally, the report highlighted the following warnings:

- The site is served over HTTP and does not redirect to HTTPS, making it vulnerable to man-in-the-middle attacks.
- The JSESSIONID cookie is set without the Secure flag, further exposing it to interception over unencrypted connections.
- The Server header exposes the specific software and version (can aid attackers in identifying known vulnerabilities.

3. Vulnerability Assessment Findings

3.1 Broken Authentication and Session Management

- **Risk Level:** Medium
- **Location:** Application-wide (session cookie handling).
- **Description:** The JSESSIONID cookie, used for session management, is transmitted over unencrypted HTTP and lacks the Secure flag. This makes it susceptible to interception by attackers performing man-in-the-middle attacks.
- **Impact:** Attackers can hijack user sessions, gaining unauthorized access to user accounts without needing their credentials.
- **Remediation:** Enforce HTTPS for all communications to protect data in transit. Set the Secure flag on all session cookies to ensure they are only transmitted over encrypted connections

3.2 Security Misconfiguration (Missing Security Headers) Risk Level:

Low to Medium Location: Web server configuration.

Description: As identified during reconnaissance, the web server is missing several crucial security headers, weakening the application's overall security posture.

Impact: Increased vulnerability to various client-side attacks, including clickjacking, XSS, and MIME-sniffing. Lack of these headers makes it easier for attackers to exploit other vulnerabilities.

Remediation: Configure the web server (e.g., Apache Tomcat) to include all recommended security headers, such as Content-Security-Policy , X-FrameOptions , X-Content-Type-Options ,Referrer-Policy , and PermissionsPolicy .

3.3 SSL/TLS Vulnerabilities

3.3.1. Obsolete Protocol Support (TLS 1.0 and TLS 1.1)

Risk Level: High

Location: Server SSL/TLS configuration.

Description: The server supports TLS 1.0 and TLS 1.1, both of which are officially deprecated by the Internet Engineering Task Force (IETF) [5] due to known security weaknesses. These protocols are vulnerable to several wellknown cryptographic attacks (e.g., BEAST, POODLE) and fail to meet modern security standards.

Impact: Continued use of these protocols leads to non-compliance with industry standards such as PCI DSS and NIST guidelines [6], and exposes sensitive data to potential interception and decryption by attackers.

Remediation: Immediately disable support for TLS 1.0 and TLS 1.1. Configure the server to prioritize and exclusively use TLS 1.2 and TLS 1.3.

3.3.2. Weak Diffie-Hellman (DH) Key Exchange (1024-bit)

Risk Level: High

Location: Server SSL/TLS configuration.

Description: The server utilizes 1024-bit Diffie-Hellman parameters for key exchange. This parameter size is considered weak and insecure against modern computational capabilities.

Impact: This vulnerability makes the connection susceptible to passive eavesdropping and potential decryption of past communications if the DH parameters are compromised, as demonstrated by the Logjam attack . It undermines the forward secrecy of the connection.

Remediation: Increase the strength of Diffie-Hellman parameters to at least 2048 bits, preferably 4096 bits, to mitigate risks associated with weak DH key exchange.

4. Conclusion

The demo.testfire.net application exhibits several significant security vulnerabilities, including missing security headers, and critical SSL/TLS weaknesses. The SSL/TLS configuration, in particular, shows a severe lack of adherence to modern security best practices, with support for obsolete protocols and weak cryptographic parameters. Addressing these issues is crucial to protect user data and maintain the integrity and availability of the application.

5. Recommendations

It is strongly recommended to implement the suggested remediation steps for each identified vulnerability. Prioritize fixing the high-risk vulnerabilities (Obsolete TLS Protocols, and Weak DH Key Exchange) immediately, followed by improvements to session management, security header configurations, and overall SSL/TLS hardening. Regular security audits and penetration testing should be conducted to ensure ongoing security.